MASTER'S MODULE · AGENTIC AI · WINTER SEMESTER

Foundations & Classical Agents

What makes software an agent — from Russell and Norvig's percept-action mapping to today's LLM agents, and how to tell real autonomy from marketing.

Unit 01 · Session 01 · Week 1

Prof. Dr. Hauke Schramm · Department of Computer Science · Kiel University

Today's agenda

1. **Course logistics & roadmap** ECTS, exam, weekly lab, the nine units
2. **What is an agent?** Classical definitions: percepts, actions, rationality, environments
3. **The modern LLM agent** Model + tools + loop + goal — a working definition
4. **Agents, workflows, chatbots** The autonomy spectrum and agent-washing
5. **Classical architectures** Reactive control, subsumption, BDI
6. **What carries over** Old ideas inside new agents; this week's lab

Course logistics

Module infAgAI-01a · 6 ECTS · MSc Computer Science · taught in English

Format — 2 SWS lecture plus 2 SWS lab, every week of the semester

Assessment — 120-minute written exam; oral examination if the cohort is small

Prerequisite — successful lab participation is required for exam admission

Lab — weekly gap-fill notebooks; agents run on a local Ollama server, no API keys

Materials — slides, video script and ~50 questions with answers per session

WHY LOCAL MODELS

Labs use locally hosted, tool-capable open-weight models via Ollama (default qwen2.5:7b). Local inference avoids provider charges; hardware, electricity, memory and throughput remain constraints. Open weights are not open training data: check licences. Cloud-API examples on slides are labelled as such.

The course in nine units

Fourteen weeks, one storyline: from a bare model call to a secured, evaluated agent system

U1 · Foundations

W1: agent definitions, autonomy, classical architectures.

U2 · Agent Loop

W2: the minimal loop, four design patterns.

U3 · Tools & MCP

W3: function calling, Model Context Protocol.

U4 · Prompting

W4-5: prompt engineering, reasoning models.

U5 · Orchestration

W6-7: ReAct, plan-and-execute, frameworks vs. code.

U6 · Memory & RAG

W8-9: memory taxonomy, vector search, retrieval.

U7 · Guardrails

W10-11: sandboxing, observability, tracing.

U8 · Multi-Agent

W12: agent teams, self-reflection.

U9 · Security & Eval

W13-14: prompt injection, evaluation, accountability.

What we will build in the labs

Fourteen gap-fill notebooks (90-120 min each), all on a local Ollama model with offline data and numbered report tasks

Labs 01-02 · Baseline

Ollama setup, bare completions, a rule-based reflex agent for source triage; then the minimal loop around ollama.chat, stop conditions, token instrumentation.

Labs 03-05 · Tools, prompts

Function calling and a small MCP server; structured output and prompt robustness; reasoning models, inference-time compute and model routing.

Labs 06-07 · Orchestration

ReAct, plan-and-execute and DAG pipelines; then a build-it-yourself mini-framework to see what frameworks really add.

Labs 08-09 · Memory, RAG

Memory taxonomy, compaction, persistent memory files; embeddings, chunking and a plain-NumPy RAG pipeline over an offline corpus.

Labs 10-11 · Safety, tracing

Action policies, approval gates, kill switch and red-teaming; traces, spans, structured logging and debugging from traces.

Labs 12-14 · Teams, eval

Reflection loops and orchestrator/critic teams; prompt injection attack and defence; an eval harness with LLM-as-judge and footprint.

1

What is an agent?

Fifty years of definitions before the LLM arrived

The classical definition (Russell & Norvig)

Artificial Intelligence: A Modern Approach — the standard vocabulary since 1995

Agent — anything mapping percept sequences to actions in an environment

Agent function — abstract mapping from percept history to action — the specification

Agent program — concrete implementation of that function on an architecture

Rational agent — acts to maximise expected performance, given the evidence so far

PEAS — Performance, Environment, Actuators, Sensors — the task template

Autonomy — behaviour driven by own percepts, not only designer assumptions

AIMA, CH. 2

“An agent is anything that can be viewed as perceiving its environment through sensors and acting upon that environment through actuators.”

Task environments shape agent design

Six of Russell & Norvig's environment dimensions (AIMA lists seven), applied to the open web as seen by a web-research assistant

Dimension	Question it asks	The open web, for a web-research assistant
Observable?	Do percepts reveal the full relevant state?	Partially: one result list or page at a time
Deterministic?	Do current state and action uniquely determine the next state?	Depends on the environment model; changing results alone prove nothing
Episodic?	Are decisions independent of earlier percepts?	Sequential: each search builds on earlier findings
Static?	Does the world change while the agent deliberates?	Dynamic: content changes while the agent works
Discrete?	Are states, time, percepts, actions discrete or continuous?	Discrete actions and text percepts; unbounded, not finite
Single-agent?	Do other agents act strategically on the outcome?	SEO operators may act strategically; a paywall is an access condition

Partially observed, changing and sequential for sure; stochasticity and strategic actors depend on the environment model you adopt.

Four control architectures, plus learning

Russell & Norvig's agent programs: increasing internal structure in four control architectures; learning is an orthogonal capability

Simple reflex

Condition-action rules on the current percept only. Fails wherever history matters (Lab 01).

Model-based reflex

Adds internal state that tracks the unobserved parts of the world.

Goal-based

Predicts which future states satisfy an explicit goal; plans toward them.

Utility-based

Ranks outcomes by a utility function; trades off conflicting goals.

Learning (cross-cutting)

Any of the four can improve its components from experience. Classical agents already learned policies; this is not a fifth rung.

The modern definition: LLM agents

Four components, their shared state, and one working definition for this course

Model — a large language model acts as the policy — it picks the next step

Tools — functions the model may invoke: search, code execution, files, APIs

Loop — call, act, observe, repeat — until a stop condition fires

Goal — a natural-language objective; success criteria often only implicit

State — the context is the model-visible working state; runtime and environment state exist too

WORKING DEFINITION

An agent is a system in which an LLM decides, in a loop, which actions to take next in pursuit of a goal — calling tools and observing their results.

2

Agents, workflows, chatbots

The autonomy spectrum — and how to spot agent-washing

Workflows vs. agents (Anthropic, 2024)

"Building Effective Agents": the engineering distinction this course adopts — not a refutation of the classical definition

Code-directed workflow

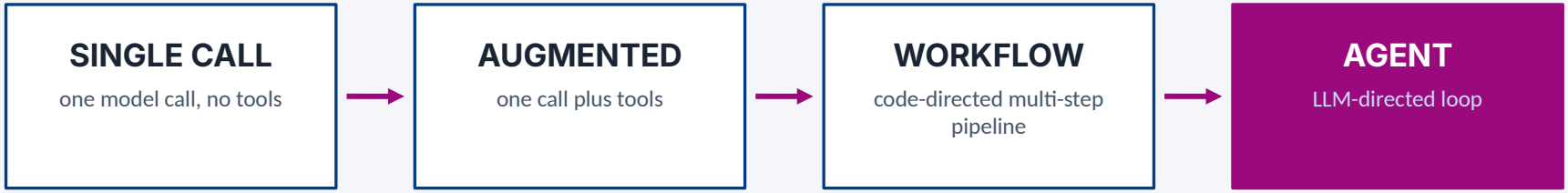
- control flow fixed in code at design time
- LLM fills slots inside predefined steps
- predictable cost, latency, failure surface
- testable like ordinary software
- right choice for well-understood tasks

LLM-directed agent

- LLM decides the next step at runtime
- path through the tools emerges per task
- handles open-ended, variable-step problems
- cost and trajectory vary run to run
- needs guardrails, budgets, observability

A spectrum of autonomy, not a binary

Four simplified architecture examples; each hands more control-flow decisions from developer to model



- Simplified examples, not a universal autonomy scale. Chat is an interface: it can expose any of these architectures.
- Autonomy is a dial the designer sets. Approval gates constrain authority without removing model-directed control.
- More autonomy buys flexibility; it costs predictability, money and safety assurances.

Agent-washing

“Over 40% of agentic AI projects will be cancelled by the end of 2027.”

Gartner, June 2025. The same analysis judged that of thousands of vendors advertising “agentic AI”, only about 130 offered real agents — the rest rebrand chatbots, RPA pipelines and rule engines. Inflated claims are now meeting real procurement budgets.

An agent-washing checklist

Five questions to ask of any system marketed as an "AI agent"

Control flow — who picks the next step at runtime — the model, or a script?

Actions — does it execute tools and observe results, or only generate text?

Loop — can the number of steps vary with the task, under stop conditions?

Recovery — does it replan after a failed action, or simply abort?

Claims vs. reality — do the autonomy claims match the deployed control flow? Approval gates limit authority, not agency

RULE OF THUMB

If the path through the system is fixed in code at design time, this course calls it a workflow — however the landing page describes it. Often that is the better engineering choice.

3

Classical agent architectures

Deliberation, reaction, subsumption and BDI — roughly 1970 to 1995

Three architecture families

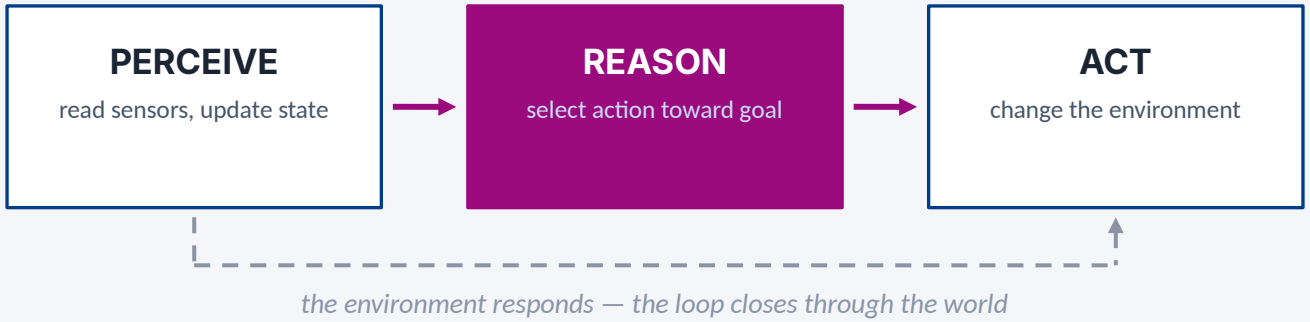
How pre-LLM AI answered the question: where does the next action come from?

Family	Control regime	World model	Canonical system
Deliberative	sense, plan, then act	explicit symbolic model	Shakey / STRIPS (1971)
Reactive	direct percept-action coupling	no central symbolic model (Brooks)	Brooks' subsumption robots (1986)
Hybrid / BDI	deliberate, commit, react	beliefs plus plan library	PRS (Georgeff & Lansky, 1987)
LLM agent	model-directed loop	context as model-visible working state	this course, Unit 2 onward

The recurring tension: deliberation is powerful but slow; reaction is fast but short-sighted.

The perceive-reason-act loop

The oldest control structure in agent design — and still the newest



- Deliberative agents spend most of their time in REASON; reactive agents couple percepts to actions directly, with minimal deliberation.
- From Unit 2 on, the same loop returns with an LLM sitting in the REASON box.

Subsumption: the reactive rebellion

Rodney Brooks, MIT, 1986-1991 — "intelligence without representation"

Layered behaviours — avoid collisions, wander, explore — each layer fully competent

Subsumption — higher layers suppress or override lower ones; no central executive

No central world model — direct sensor-actuator coupling instead of a central symbolic model; layers may keep local state

Result — robots that roamed offices in real time while planners timed out

Limit — hard to scale toward abstract, long-horizon goals

BROOKS, 1990

"The world is its own best model." Competence, he argued, comes from interaction with the environment — not from reasoning over an internal symbolic copy of it.

BDI: beliefs, desires, intentions

Bratman's practical reasoning (1987), engineered into PRS (Georgeff & Lansky, 1987; Rao & Georgeff, 1995)

Beliefs

The agent's current — possibly wrong — information about the world, updated from percepts.

Desires

Goal states the agent would like to bring about; they may conflict.

Intentions

Desires the agent has committed to; they persist and constrain replanning.

Plan library

Hand-written recipes; deliberation selects one for each adopted intention.

Deliberation cycle

Update beliefs, reconsider goals, execute the next plan step — every tick.

Commitment

Reconsider intentions rarely: stability versus reactivity is a tunable trade-off.

A classical agent loop, in skeleton form

Illustrative BDI skeleton: belief revision, plan applicability, completion and failure handling are omitted

bdi_skeleton.py

```
class BDIAgent:
    def __init__(self, plan_library):
        self.beliefs = {}          # world model
        self.intentions = []       # committed plan steps
        self.plans = plan_library  # hand-written recipes

    def step(self, percept):
        self.beliefs.update(percept)           # perceive
        if self.should_reconsider() or not self.intentions:
            goal = self.choose_desire(self.beliefs)
            self.intentions = list(self.plans.get(goal, []))
        if not self.intentions:          # no applicable plan
            return NOOP               # explicit failure path
        return self.intentions.pop(0)          # act
# Skeleton only. The intelligence lives in choose_desire,
# the plan library and should_reconsider -- all hand-written.
```

4

From classical to LLM agents

What survives, what is genuinely new

What survives — and what is new

LLM agents inherit the skeleton of classical designs, not their machinery

Survives in LLM agents

- the sense-think-act loop as control skeleton
- intention persistence: commit, don't replan constantly
- layering: fast checks beneath slow deliberation
- working state (the context) kept separate from the policy
- learned policies and generated plans: classical agents had both

New with LLM agents

- broadly pretrained language models as reusable policies and interfaces
- state and goals expressed in natural language
- tool improvisation: unforeseen action sequences
- one model generalises across many domains
- new failure modes: hallucination, prompt injection

Lab 01's source-triage task, in classical vocabulary

Applying today's concepts to this week's reflex agent: 20 search records, one KEEP or DISCARD decision each

PEAS — performance: accuracy against gold labels; environment: a CSV of search records; actuators: KEEP, DISCARD; sensors: domain, title, snippet

Environment class — per record fully observed, episodic and static; the tricky subset shows where history and adversaries matter

BDI reading — beliefs: the current percept only; desires: keep trustworthy, relevant sources; intentions: none persist between calls

State — no model-visible working state yet; Lab 02 adds the context, beside runtime and environment state

Open question — where do plans come from? Libraries, symbolic search, learned policies or LLM generation

LAB 01, THIS WEEK

Part A: Ollama check. B: bare completions, system prompt, temperature. C: hand-coded reflex agent. D: where its rules break. E: one LLM call as learned policy. F: PEAS and BDI write-up. G: tuning. Report tasks R1-R3.

TAKEAWAYS

What you should remember

- **Agent —** percepts mapped to actions in an environment — the definition predates LLMs by decades
- **LLM agent —** an LLM deciding, in a loop, which tools to call in pursuit of a goal
- **Spectrum —** single call, augmented LLM, workflow, agent — simplified examples; autonomy is a design dial
- **Agent-washing —** check control flow, actions, loop and recovery before believing any label
- **Classics —** subsumption gave us layering, BDI gave us intention persistence — both survive

NEXT SESSION The agent loop — LLM, tool calls, observation, context buffer, stop conditions.

Reading & references

- Russell, S. & Norvig, P. (2020). Artificial Intelligence: A Modern Approach, 4th ed. Pearson.
  Chapter 2, Intelligent Agents — the core reading for this session.
- Wooldridge, M. & Jennings, N. R. (1995). Intelligent Agents: Theory and Practice. The Knowledge Engineering Review, 10(2), 115-152.
- Brooks, R. A. (1991). Intelligence without Representation. Artificial Intelligence, 47(1-3), 139-159.
  See also Brooks (1990), Elephants Don't Play Chess, for "the world is its own best model".
- Rao, A. S. & Georgeff, M. P. (1995). BDI Agents: From Theory to Practice. Proc. ICMAS-95.
  Philosophical basis: Bratman (1987), Intention, Plans, and Practical Reason.
- Anthropic (2024). Building Effective Agents. Anthropic engineering blog, December 2024.
  Source of the workflow/agent distinction used throughout this course.
- Wang, L. et al. (2024). A Survey on Large Language Model Based Autonomous Agents. Frontiers of Computer Science, 18(6).
- Gartner (2025). Press release, June 2025: over 40 percent of agentic AI projects to be cancelled by end of 2027.